MarketSight

Stock Market Prediction System

Sprint 5 - Construction 3

Project Deliverables

Team Members: Zane Tessmer (Scrum Master) Harsh Kumar (Product Owner / Lead Developer) Dante Bertolutti (Developer) Parag Jindal (Developer) Kaval S (Developer) Guntash (Developer)

Date: **March 17, 2026** Course: COSC 471 - Winter 2026 Okanagan College

Table of Contents

Part A - Sprint Meetings & Planning

1. Sprint Retrospective (Sprint 4 Closing)
2. Sprint Review Meeting Notes
3. Sprint Planning Meeting Notes
4. Backlog Grooming Meeting Notes
5. Project Backlog
6. Sprint Backlog
7. Individual User Stories & Tasks

Part B - Technical Documentation 8. Final Business Case 9. Final Software Specification 10. Final Use Case Specifications 11. Final Configuration Management Plan 12. Change Request Document 13. User Support Documentation 14. Developer's Guide (Final)

Part C - Testing & Acceptance 15. Acceptance Test Results

Part A - Sprint Meetings & Planning

1. Sprint Retrospective (Sprint 4 Closing)

Meeting Date: March 4, 2026 **Duration:** 15 minutes **Facilitator:** Zane Tessmer (Scrum Master)
Attendees: Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash
Template: Confluence Retrospectives

What Went Well

- Harsh successfully migrated the entire data pipeline from yfinance to FMP API with 53-feature engineering
- Optuna HPO significantly improved model training quality on DRAC cluster
- Full end-to-end pipeline (data -> features -> model -> API -> Airflow -> frontend) completed
- Dante implemented prediction confidence scoring and built the predictions page UI
- Streamlit frontend added as an alternative interface for data exploration
- PR #48 (ML training pipeline) and PR #49 (DRI migration + Streamlit) merged successfully
- Sprint velocity hit 73 points, significantly above Sprint 3

What Didn't Go Well

- Harsh's ML pipeline merge (PR #48) overwrote Dante's confidence scoring code and predictions page (PR #45 still open)
- Communication breakdown about conflicting changes to inference module between branches
- No merge coordination meeting held before major refactors
- Predictions page on main is now a stub - lost significant frontend work
- Some open issues from Sprint 4 (#28, #25, #27) still unresolved
- XP pair rotation did not happen as planned - most pairs stayed the same

Action Items

Action Item	Owner	Due Date

Rebase dante_feature onto main and resolve conflicts	Dante Bertolutti	Mar 7, 2026
Re-integrate confidence scoring with new 53-feature pipeline	Dante Bertolutti	Mar 10, 2026
Hold merge coordination meeting before major refactors	Zane Tessmer	Ongoing
Rotate XP pairs for Sprint 5	Zane Tessmer	Mar 5, 2026
Set up branch protection rules to prevent overwriting	Harsh Kumar	Mar 6, 2026

Documented by: Zane Tessmer

2. Sprint Review Meeting Notes

Sprint Review Period: March 4 - March 17, 2026 **Duration:** 15 minutes **Facilitator:** Zane Tessmer (Scrum Master) **Attendees:** Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash **Template:** Confluence Meeting Notes

Sprint 5 Goal

Reach Initial Operational Capability (IOC): complete the production-ready MVP with restored prediction features, server migration, real data integration, and acceptance testing for the final release demo.

Closed Issues

Issue	Title	Closed Date
#34	Create feature backfill job for 2yr history	March 4, 2026
#26	Build useOHLC hook - fetch candle data for selected stock	March 4, 2026

Merged Pull Requests (This Sprint)

PR	Title	Author	Merged Date
#49	Add Streamlit frontend and direct server setup	Harsh Kumar	March 12, 2026
#48	Ft ml training (53-feature pipeline, Optuna HPO, DRAC)	Harsh Kumar	March 4, 2026

Open Pull Requests

PR	Title	Author	Status
#45	feat: Add confidence scores, build predictions page, and align frontend types	Dante Bertolutti	Open (needs rebase)

Completed Work

User Story / Feature	Developer	Status
US-30: Rebase and re-integrate confidence scoring	Dante Bertolutti	Completed
US-31: Restore predictions page with new pipeline	Dante Bertolutti	Completed
US-32: Streamlit frontend for data exploration	Harsh Kumar	Completed

US-33: DRI server migration and setup scripts	Harsh Kumar	Completed
US-34: Native XGBoost model format migration	Harsh Kumar	Completed
US-35: Feature engineering module refactor	Harsh Kumar	Completed
US-36: Data warehouse sync scripts	Harsh Kumar	Completed
US-37: Stock detail page UI improvements	Harsh Kumar	Completed
US-38: Frontend datetime UTC standardization	Harsh Kumar	Completed
US-39: Sprint documentation and deliverables	Zane Tessmer	Completed
US-40: Final acceptance testing	Zane Tessmer & Dante Bertolutti	Completed
US-41: Frontend API client cleanup	Parag Jindal	Completed
US-42: Dashboard performance optimization	Kaval S	Completed
US-43: UI component polish and accessibility	Guntash	Completed

Demo Summary

The team demonstrated the final release to the client and professor. Harsh showed the production server running on DRI infrastructure with real FMP market data, the Streamlit data exploration tool, and the native XGBoost model serving pipeline. Dante demonstrated the restored prediction confidence scoring system working with the 53-feature pipeline and the full predictions page. The acceptance test was signed off by the client.

Stakeholder Feedback

- Client approved the system for initial operational capability
- Professor confirmed the demo met Construction phase exit criteria
- Client requested prediction history tracking as a future enhancement

Documented by: Zane Tessmer

3. Sprint Planning Meeting Notes

Meeting Date: March 4, 2026 **Duration:** 20 minutes **Facilitator:** Zane Tessmer (Scrum Master)
Attendees: Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash

Part 1 - Product Backlog Prioritization (10 min)

The team reviewed the Product Backlog with Harsh (Product Owner) and prioritized items for Sprint 5 based on the IOC milestone requirements and the need to resolve merge conflicts from Sprint 4.

Priority order agreed upon:

1. Rebase dante_feature and re-integrate confidence scoring (HIGH - blocked code)
2. Restore predictions page UI with new pipeline compatibility (HIGH - core feature lost)
3. DRI server migration and production setup (HIGH - deployment requirement)
4. Native XGBoost model format migration (HIGH - model serving)
5. Feature engineering module refactor (HIGH - inference accuracy)
6. Streamlit frontend for data exploration (MEDIUM - secondary interface)
7. Data warehouse sync scripts (MEDIUM - data freshness)
8. Stock detail UI improvements (MEDIUM - polish)
9. Frontend datetime standardization (LOW - consistency)
10. Final acceptance testing and documentation (HIGH - milestone gate)

Part 2 - Sprint Backlog Planning (10 min)

Sprint 5 Duration: March 4 - March 17, 2026 (1 week + buffer)

Sprint Goal: Achieve IOC milestone - deliver a production-ready MVP with all features restored, real data, and signed acceptance tests.

XP Pairs for Sprint 5:

Pair	Members	Focus Area
Pair 1	Harsh Kumar & Dante Bertolutti	ML pipeline integration + confidence scoring rebase
Pair 2	Parag Jindal & Kaval S	Frontend cleanup + dashboard optimization

Pair 3	Guntash & Zane Tessmer	UI polish + documentation + acceptance testing

Capacity Planning:

Team Member	Available Hours	Allocated Stories
Harsh Kumar	50 hrs	US-32, US-33, US-34, US-35, US-36, US-37, US-38
Dante Bertolutti	30 hrs	US-30, US-31, US-40
Parag Jindal	20 hrs	US-41
Kaval S	15 hrs	US-42
Guntash	15 hrs	US-43
Zane Tessmer	25 hrs	US-39, US-40 (documentation & acceptance testing)

Total Story Points committed: 62 points

Documented by: Zane Tessmer

4. Backlog Grooming Meeting Notes

Meeting Date: March 5, 2026 **Duration:** 20 minutes **Facilitator:** Zane Tessmer (Scrum Master)
Attendees: Zane Tessmer, Harsh Kumar, Dante Bertolutti, Parag Jindal, Kaval S, Guntash

Estimation Results (Story Points)

User Story	Title	Points	Notes
US-30	Rebase and re-integrate confidence scoring	5	Adapt confidence.py to 53-feature pipeline, resolve merge conflicts
US-31	Restore predictions page with new pipeline	5	Rebuild predictions UI for updated inference API
US-32	Streamlit frontend for data exploration	5	New Streamlit app for market data, inference, datasets
US-33	DRI server migration and setup scripts	8	Server docs, management scripts, SSH tunnel setup
US-34	Native XGBoost model format migration	5	Switch from JSON to native .ubj format with meta.json
US-35	Feature engineering module refactor	8	Centralize feature calc, integrate into inference service
US-36	Data warehouse sync scripts	5	PostgreSQL remote-to-local sync with SSH tunnel
US-37	Stock detail page UI improvements	3	Remove stock list table, streamline layout, chart range breaks
US-38	Frontend datetime UTC standardization	2	Centralize SQL queries, standardize datetime display
US-39	Sprint documentation and deliverables	3	Meeting notes, backlogs, reports, final docs
US-40	Final acceptance testing	5	End-to-end acceptance test suite, client sign-off
US-41	Frontend API client cleanup	3	TypeScript types matching backend schemas, hook cleanup
US-42	Dashboard performance optimization	3	Loading state improvements, query optimization
US-43	UI component polish and accessibility	2	Consistent styling, keyboard navigation, contrast

Acceptance Criteria Refinements

US-30 (Confidence Scoring Rebase):

- Confidence score must work with the new 53-feature pipeline
- Score must still be between 0.0 and 1.0
- Must pass updated unit tests against new feature names
- Inference service must return confidence in PredictionResponse (not None)

US-31 (Predictions Page Restoration):

- Stock selector dropdown with all active stocks
- Individual predict and "Predict All" functionality
- Prediction cards with confidence display and color coding
- "View Chart" button linking to stock detail page

US-40 (Acceptance Testing):

- All 15+ test cases must pass
- Client must sign the acceptance test document
- System must be demonstrated to professor during project meeting

Documented by: Zane Tessmer

5. Project Backlog

ID	User Story	Priority	Points	Status	Sprint
US-01	Set up project repository and Docker environment	HIGH	5	Done	Sprint 3
US-02	Implement XGBoost model training notebook	HIGH	8	Done	Sprint 3
US-03	Create PostgreSQL database schema	HIGH	5	Done	Sprint 3
US-04	Build initial frontend scaffold with React 19	HIGH	5	Done	Sprint 3
US-05	Set up CI/CD pipeline with GitHub Actions	HIGH	5	Done	Sprint 3
US-06	Create market data seeding scripts	MEDIUM	3	Done	Sprint 3
US-07	Design system architecture diagrams	MEDIUM	2	Done	Sprint 3
US-08	Write project README and development guide	LOW	2	Done	Sprint 3
US-11	Build inference API endpoint	HIGH	8	Done	Sprint 4
US-12	Create stock chart with prediction overlay	HIGH	5	Done	Sprint 4
US-13	Implement market module (backend)	HIGH	8	Done	Sprint 4
US-14	Add prediction confidence scoring	HIGH	5	Done	Sprint 4
US-15	Build predictions page UI	HIGH	5	Done	Sprint 4
US-16	Set up Airflow DAGs	MEDIUM	5	Done	Sprint 4
US-17	Configure Nginx reverse proxy	MEDIUM	2	Done	Sprint 4
US-18	Write backend test suite	MEDIUM	5	Done	Sprint 4
US-19	Refactor frontend routing	MEDIUM	3	Done	Sprint 4
US-20	Build admin user management page	MEDIUM	3	Done	Sprint 4
US-21	Update user settings page	LOW	2	Done	Sprint 4
US-22	Frontend dashboard summary cards	LOW	2	Done	Sprint 4
US-23	Update project documentation	LOW	3	Done	Sprint 4
US-24	Write SW specification document	LOW	3	Done	Sprint 4
US-25	Create sprint deliverable documents	LOW	3	Done	Sprint 4
US-26	FMP data integration (replace yfinance)	HIGH	8	Done	Sprint 4
US-27	53-feature engineering pipeline	HIGH	5	Done	Sprint 4
US-28	XGBoost Optuna HPO training pipeline	HIGH	8	Done	Sprint 4
US-29	DRAC end-to-end training pipeline	HIGH	5	Done	Sprint 4
US-30	Rebase and re-integrate confidence scoring	HIGH	5	Done	Sprint 5

US-31	Restore predictions page with new pipeline	HIGH	5	Done	Sprint 5
US-32	Streamlit frontend for data exploration	MEDIUM	5	Done	Sprint 5
US-33	DRI server migration and setup scripts	HIGH	8	Done	Sprint 5
US-34	Native XGBoost model format migration	HIGH	5	Done	Sprint 5
US-35	Feature engineering module refactor	HIGH	8	Done	Sprint 5
US-36	Data warehouse sync scripts	MEDIUM	5	Done	Sprint 5
US-37	Stock detail page UI improvements	MEDIUM	3	Done	Sprint 5
US-38	Frontend datetime UTC standardization	LOW	2	Done	Sprint 5
US-39	Sprint documentation and deliverables	LOW	3	Done	Sprint 5
US-40	Final acceptance testing	HIGH	5	Done	Sprint 5
US-41	Frontend API client cleanup	MEDIUM	3	Done	Sprint 5
US-42	Dashboard performance optimization	LOW	3	Done	Sprint 5
US-43	UI component polish and accessibility	LOW	2	Done	Sprint 5
US-44	Implement WebSocket real-time streaming	HIGH	8	Pending	Backlog
US-45	Add JWT authentication	HIGH	8	Pending	Backlog
US-46	Implement prediction history storage	MEDIUM	5	Pending	Backlog
US-47	Add frontend unit tests with Vitest	MEDIUM	5	Pending	Backlog
US-48	Restrict CORS for production	LOW	2	Pending	Backlog

6. Sprint 5 Backlog

Sprint Duration: March 4 - March 17, 2026 **Sprint Goal:** Achieve IOC milestone with production-ready MVP **Total Story Points:** 62 | **Completed:** 62 | **Velocity:** 62

ID	User Story	Owner	Points	Status	Hours Est.	Hours Actual
US-30	Rebase and re-integrate confidence scoring	Dante Bertolutti	5	Done	8	9
US-31	Restore predictions page with new pipeline	Dante Bertolutti	5	Done	8	7
US-32	Streamlit frontend for data exploration	Harsh Kumar	5	Done	6	8
US-33	DRI server migration and setup scripts	Harsh Kumar	8	Done	12	14
US-34	Native XGBoost model format migration	Harsh Kumar	5	Done	6	5
US-35	Feature engineering module refactor	Harsh Kumar	8	Done	10	12
US-36	Data warehouse sync scripts	Harsh Kumar	5	Done	6	7
US-37	Stock detail page UI improvements	Harsh Kumar	3	Done	4	3
US-38	Frontend datetime UTC standardization	Harsh Kumar	2	Done	2	2
US-39	Sprint documentation and deliverables	Zane Tessmer	3	Done	5	6
US-40	Final acceptance testing	Zane Tessmer & Dante	5	Done	6	7
US-41	Frontend API client cleanup	Parag Jindal	3	Done	4	4
US-42	Dashboard performance optimization	Kaval S	3	Done	4	3
US-43	UI component polish and accessibility	Guntash	2	Done	3	3

Burndown Summary

- **Week 1 (Mar 4-10):** 41 points completed (US-30, US-32, US-33, US-34, US-35, US-36, US-41, US-42)

- **Week 2 (Mar 11-17):** 21 points completed (US-31, US-37, US-38, US-39, US-40, US-43)

7. Individual User Stories & Tasks

Dante Bertolutti

US-30: Rebase and re-integrate confidence scoring

> As a developer, I want to rebase my confidence scoring feature onto the updated main branch so that predictions include reliability scores with the new 53-feature pipeline.

Task	Description	Hours	Status
T-30.1	Rebase dante_feature branch onto origin/main	2	Done
T-30.2	Resolve merge conflicts in inference module	2	Done
T-30.3	Adapt confidence.py to new feature names and pipeline	2	Done
T-30.4	Update unit tests for confidence scoring	1.5	Done
T-30.5	Integration test: confidence scores in prediction response	1	Done
T-30.6	Code review with Harsh on inference service changes	0.5	Done

US-31: Restore predictions page with new pipeline

> As a user, I want the full predictions page back so that I can generate and view forecasts for individual stocks or all stocks at once.

Task	Description	Hours	Status
T-31.1	Rebuild predictions.tsx with stock selector component	2	Done
T-31.2	Implement PredictionCard with updated PredictionResponse type	2	Done
T-31.3	Add "Predict All" batch functionality	1	Done
T-31.4	Add color-coded confidence indicators (green/yellow/red)	0.5	Done
T-31.5	Update useInference hook for new API response shape	1	Done
T-31.6	Manual testing: single predict, batch predict, error states	0.5	Done

US-40: Final acceptance testing (shared with Zane)

> As the team, we want to run comprehensive acceptance tests so that the client can sign off on the IOC milestone.

Task	Description	Hours	Status

T-40.1	Execute all acceptance test cases on production build	2	Done
T-40.2	Document test results with screenshots	1.5	Done
T-40.3	Fix any failing acceptance criteria	1	Done

Harsh Kumar

US-32: Streamlit frontend for data exploration

> As a data analyst, I want a Streamlit interface so that I can explore market data, run inference, and manage dataset snapshots outside the main React frontend.

Task	Description	Hours	Status
T-32.1	Create Streamlit app structure with page navigation	2	Done
T-32.2	Implement market data exploration page	2	Done
T-32.3	Add inference prediction page with real-time results	2	Done
T-32.4	Build dataset snapshot management page	2	Done

US-33: DRI server migration and setup scripts

> As a developer, I want the application deployed on the DRI production server with proper setup documentation so that we have a stable production environment.

Task	Description	Hours	Status
T-33.1	Write server setup documentation (docs_server_setup.md)	3	Done
T-33.2	Create management scripts for service lifecycle	3	Done
T-33.3	Configure SSH tunnel for database access	2	Done
T-33.4	Set up Nginx with SSL for production	2	Done
T-33.5	Validate deployment on DRI infrastructure	2	Done
T-33.6	Document troubleshooting procedures	2	Done

US-34: Native XGBoost model format migration

> As a developer, I want to switch from JSON model format to native XGBoost .ubj format so that model loading is faster and artifacts are structured with meta.json.

Task	Description	Hours	Status
T-34.1	Update model_loader.py for native XGBoost format	2	Done

T-34.2	Create meta.json artifact structure	1.5	Done
T-34.3	Write script for loading final data to PostgreSQL	1.5	Done

US-35: Feature engineering module refactor

> As a developer, I want a centralized feature engineering module so that both training and inference use identical feature calculations.

Task	Description	Hours	Status
T-35.1	Create features module with technical indicator calculations	4	Done
T-35.2	Integrate feature module into inference service	3	Done
T-35.3	Optimize OHLC data retrieval with raw SQL	2	Done
T-35.4	Validate feature output matches training pipeline	3	Done

US-36: Data warehouse sync scripts

> As a developer, I want scripts to synchronize data from the remote DRI server to local PostgreSQL so that developers can work with production data locally.

Task	Description	Hours	Status
T-36.1	Write sync_dw_to_local_postgres.py with SSH tunnel support	4	Done
T-36.2	Add server setup docs with SSH tunnel instructions	2	Done
T-36.3	Test sync with production data	1	Done

US-37: Stock detail page UI improvements

> As a user, I want a cleaner stock detail page so that the layout is easier to read and charts display properly.

Task	Description	Hours	Status
T-37.1	Remove stock list table from detail page	1	Done
T-37.2	Streamline stock detail layout	1	Done
T-37.3	Add range breaks to price chart x-axis	1	Done

US-38: Frontend datetime UTC standardization

> As a developer, I want all datetime displays standardized to UTC so that timestamps are consistent across the application.

Task	Description	Hours	Status
T-38.1	Centralize stock metadata SQL queries	1	Done
T-38.2	Standardize frontend datetime display to UTC	1	Done

Zane Tessmer

US-39: Sprint documentation and deliverables

> As a Scrum Master, I want all sprint ceremonies and deliverables documented for the course submission.

Task	Description	Hours	Status
T-39.1	Write Sprint 4 Retrospective report	1	Done
T-39.2	Write Sprint Review meeting notes	1	Done
T-39.3	Write Sprint Planning meeting notes	1	Done
T-39.4	Compile project and sprint backlogs	1	Done
T-39.5	Create backlog grooming report	0.5	Done
T-39.6	Prepare final deliverable package	1.5	Done

US-40: Final acceptance testing (shared with Dante)

Task	Description	Hours	Status
T-40.4	Prepare acceptance test document for client sign-off	2	Done
T-40.5	Coordinate demo with professor	1	Done

Parag Jindal

US-41: Frontend API client cleanup

> As a developer, I want the frontend API client layer to have proper TypeScript types matching the backend schemas so that the code is type-safe and maintainable.

Task	Description	Hours	Status
T-41.1	Audit frontend types against current backend Pydantic schemas	1.5	Done
T-41.2	Update TypeScript interfaces for inference and market modules	1.5	Done
T-41.3	Clean up unused hooks and service functions	1	Done

Kaval S

US-42: Dashboard performance optimization

> As a user, I want the dashboard to load faster with better loading states so that the application feels responsive.

Task	Description	Hours	Status
T-42.1	Optimize summary card data fetching with parallel queries	1.5	Done
T-42.2	Improve skeleton loading states for better UX	1	Done
T-42.3	Add error boundary for failed API calls	0.5	Done

Guntash

US-43: UI component polish and accessibility

> As a user, I want consistent styling and keyboard navigation so that the application is accessible and polished.

Task	Description	Hours	Status
T-43.1	Audit color contrast and fix accessibility issues	1	Done
T-43.2	Add keyboard navigation support to interactive elements	1	Done
T-43.3	Ensure consistent spacing and typography across pages	1	Done

Part B - Technical Documentation

8. Final Business Case

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Project Overview

MarketSight is a stock market prediction system developed as a capstone project for COSC 471 at Okanagan College. The system uses machine learning (XGBoost with Optuna hyperparameter optimization) to predict stock prices based on historical OHLCV data and 53 technical indicators sourced from the Financial Modeling Prep (FMP) API.

Business Objectives

1. Provide retail investors with ML-powered stock price predictions with confidence scores
2. Automate data ingestion from FMP API, feature engineering, and model retraining via DRAC HPC
3. Deliver an intuitive web dashboard (React) and data exploration tool (Streamlit) for interaction
4. Deploy on DRI production infrastructure with proper server management

Target Users

- Retail investors seeking data-driven insights
- Day traders who need quick price forecasts
- Finance students learning about quantitative analysis
- Data analysts exploring market data via Streamlit

Value Proposition

- **53-feature predictions** using RSI, MACD, Bollinger Bands, lag features, and more
- **Confidence scoring** to quantify prediction reliability based on market conditions
- **Dual frontends** - React dashboard for end users, Streamlit for data exploration
- **Production deployment** on DRI server with SSL and Nginx reverse proxy

- **Automated pipelines** via Airflow for data freshness and model retraining

Project Scope (Sprint 5 - Final)

The system has reached IOC with full end-to-end prediction capability, confidence scoring, production deployment on DRI infrastructure, and comprehensive acceptance testing. Authentication has been deferred to the backlog to prioritize core prediction features and production stability.

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Model accuracy needs improvement	Medium	Medium	53-feature pipeline + Optuna HPO; continued tuning
No authentication on endpoints	Medium	High	CORS restriction; JWT auth in backlog
DRI server downtime	Low	High	Management scripts + monitoring in place
Data staleness from FMP API limits	Low	Medium	Chunked downloads + gap-filling scripts
Merge conflicts between feature branches	Medium	Medium	Merge coordination meetings + branch protection

Budget & Resources

- **Infrastructure:** DRI production server (DRAC-provided), Docker-based deployment
- **Compute:** NVIDIA H100 GPU on DRAC cluster for model training
- **Data:** Financial Modeling Prep API (FMP) for historical OHLCV data
- **Team:** 6 developers, 1-2 week sprints

9. Final Software Specification

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Functional Requirements

ID	Requirement	Priority	Status
FR-01	System shall predict stock prices using XGBoost ML model (Optuna-optimized)	HIGH	Implemented
FR-02	System shall display predictions with confidence scores (0.0-1.0)	HIGH	Implemented
FR-03	System shall provide OHLC chart visualization (candlestick + line)	HIGH	Implemented
FR-04	System shall support batch prediction for all active stocks	HIGH	Implemented
FR-05	System shall ingest historical data via FMP API (replacing yfinance)	HIGH	Implemented
FR-06	System shall calculate 53 technical indicator features for inference	HIGH	Implemented
FR-07	System shall automatically seed market data daily via Airflow	MEDIUM	Implemented
FR-08	System shall retrain the ML model via Optuna HPO on DRAC cluster	MEDIUM	Implemented
FR-09	System shall provide stock metadata (symbol, name, sector, exchange)	MEDIUM	Implemented
FR-10	System shall support admin user management (add/edit/delete)	MEDIUM	Implemented
FR-11	System shall allow users to change password and delete account	LOW	Implemented
FR-12	System shall provide a Streamlit frontend for data exploration	MEDIUM	Implemented
FR-13	System shall sync data warehouse from remote to local PostgreSQL	MEDIUM	Implemented
FR-14	System shall serve models in native XGBoost .ubj format with meta.json	HIGH	Implemented
FR-15	System shall display datetime values in UTC across all frontends	LOW	Implemented
FR-16	System shall provide real-time predictions via WebSocket	HIGH	Backlog
FR-17	System shall authenticate users with JWT tokens	HIGH	Backlog
FR-18	System shall store prediction history for accuracy tracking	MEDIUM	Backlog

Non-Functional Requirements

ID	Requirement	Target	Current
NFR-01	API response time (p50)	< 100ms	~80ms
NFR-02	API response time (p95)	< 300ms	~250ms
NFR-03	Prediction inference latency	< 50ms	~35ms
NFR-04	Concurrent users supported	100+	Tested at 150

NFR-05	Backend test coverage	> 80%	~75%
NFR-06	CI pipeline execution time	< 10 min	~7 min
NFR-07	System availability	99.5%	Deployed on DRI
NFR-08	Model loading time (native format)	< 2s	~1.2s

System Constraints

- PostgreSQL 16 required for database
- Python 3.12 required for backend
- Node.js / Bun required for frontend build
- Docker and Docker Compose required for deployment
- DRAC HPC cluster access required for GPU model training
- FMP API key required for market data ingestion
- DRI server access for production deployment

Assumptions

- Market data is sourced from FMP API with chunked downloads and gap filling
- Model accuracy continues to improve with 53-feature pipeline and Optuna HPO
- Authentication is not required for the IOC demo but is critical for production
- DRI server remains available for production hosting

10. Final Use Case Specifications

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Use Case 1: Get Stock Prediction with Confidence (Dante & Harsh Pair)

Field	Description
Use Case ID	UC-01
Title	Get Stock Price Prediction with Confidence Score
Primary Actor	User
Preconditions	System is running, stock exists in database, model is loaded in native format
Postconditions	User receives predicted price with confidence score

Main Flow:

1. User navigates to the Predictions page
2. User selects a stock symbol from the dropdown
3. System sends GET request to `/api/v1/inference/predict/{symbol}`
4. Backend loads the XGBoost model (singleton, cached after first load, native .ubj format)
5. Backend retrieves latest OHLC data for the stock (minimum 150 bars)
6. Backend calculates 53 technical indicator features (RSI, MACD, Bollinger Bands, lag features, volume features, etc.)
7. Backend encodes ticker symbol using the ticker encoder
8. XGBoost model generates price prediction (predicted return)
9. Backend calculates confidence score using market condition factors
10. Backend returns PredictionResponse (symbol, current_price, predicted_price, predicted_return, prediction_date, confidence, model_version)
11. Frontend displays prediction card with color-coded confidence (green > 70%, yellow > 40%, red <= 40%)

Alternative Flows:

- **4a.** Stock not found in database: Return 404 error, display "Stock not found" message
- **5a.** Insufficient historical data (< 150 bars): Return 400 error, display "Not enough data for prediction"

- **7a.** Model error during inference: Return 500 error, display "Prediction service unavailable"
- **8a.** Batch predict ("Predict All"): Repeat steps 3-11 for all active stocks, display results in grid

Use Case 2: Explore Market Data via Streamlit (Harsh & Parag Pair)

Field	Description
Use Case ID	UC-02
Title	Explore Market Data with Streamlit Interface
Primary Actor	Data Analyst
Preconditions	Streamlit app is running, backend API is available
Postconditions	Analyst can view market data, run predictions, and manage dataset snapshots

Main Flow:

1. Analyst opens the Streamlit app in their browser
2. Analyst selects a page from the sidebar navigation (Market Data, Inference, Datasets)
3. On Market Data page: analyst browses stocks, views OHLC data, filters by sector
4. On Inference page: analyst selects a stock and runs a prediction with real-time results
5. On Datasets page: analyst builds, lists, and downloads dataset snapshot files

Alternative Flows:

- **2a.** Backend API is unavailable: Streamlit shows connection error message
- **4a.** Prediction fails: Error message displayed with reason

11. Final Configuration Management Plan

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Version Control

- **Platform:** GitHub
- **Repository:** Okanagan-College-Cosc471-Winter-2026/the-project-maverick
- **Strategy:** Feature branch workflow with PR-based code review
- **Branch naming:** `ft-<feature-name>` for feature branches
- **Main branch:** `main` (protected, requires PR review)

Branching Strategy

1. Developer creates feature branch from `main` (e.g., `ft-dri-migration`, `ft-Ml-training`)
2. Developer implements changes with atomic commits using conventional commit messages
3. Developer opens Pull Request targeting `main`
4. CI pipeline runs automatically (lint, typecheck, test, build)
5. At least one team member reviews the PR
6. Merge coordination meeting held if PR affects shared modules (inference, features)
7. After approval and passing CI, PR is merged to `main`

Active Branches (Sprint 5)

Branch	Owner	Purpose	Status
`main`	Protected	Production-ready code	Active
`dante_feature`	Dante Bertolutti	Confidence scoring + predictions page	Open PR #45
`ft-Ml-training`	Harsh Kumar	ML pipeline, FMP integration, Optuna HPO	Merged (PR #48)
`ft-dri-migration`	Harsh Kumar	DRI server setup, Streamlit, model format	Merged (PR #49)

Configuration Items

Item	Location	Owner	Change Frequency

Backend source code	`backend/app/`	Harsh Kumar	High
Frontend source code (React)	`frontend/src/`	Parag Jindal	High
Frontend source code (Streamlit)	`frontend_streamlit/`	Harsh Kumar	Medium
ML training scripts	`ml/scripts/`	Harsh Kumar	Medium
ML model artifacts	`ml/models/` (native .ubj + meta.json)	Harsh Kumar	Low (retrain cycle)
Feature engineering	`backend/app/modules/inference/features.py`	Harsh Kumar	Medium
Docker configuration	`docker-compose.yml`, `docker/`	Harsh Kumar	Low
CI/CD pipelines	`.github/workflows/`	Harsh Kumar	Low
Database migrations	`backend/alembic/versions/`	Harsh Kumar	Medium
Server setup docs	`docs_server_setup.md`	Harsh Kumar	Low
DW sync scripts	`scripts/sync_dw_to_local_postgres.py`	Harsh Kumar	Low
Environment config	`.env` (not committed)	All	Low
Documentation	`Documentation/`	Zane Tessmer	Medium

Environment Management

Environment	Purpose	Access
Local (Docker Compose)	Development and testing	All developers
CI (GitHub Actions)	Automated testing	Automated on push/PR
HPC (DRAC Fir Server)	GPU model training	Authorized team members
DRI Production Server	Production deployment	SSH access, managed via scripts

Change Control Process

1. All changes require a Pull Request
2. CI must pass before merge
3. Breaking changes to shared modules (inference, features) require team discussion
4. Merge coordination meeting before major refactors affecting multiple branches
5. Database schema changes require Alembic migration
6. Model artifact changes backed up with timestamps and meta.json versioning

12. Change Request Document

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Change Request #3

Field	Details
CR ID	CR-003
Title	Restore confidence scoring deleted during ML pipeline merge
Requested By	Dante Bertolutti (Developer)
Date Submitted	March 4, 2026
Priority	High
Status	Approved & Implemented

Description: PR #48 (ML pipeline merge) inadvertently deleted the `confidence.py` module, `test_confidence.py` tests, and gutted the predictions page UI. The confidence scoring feature and predictions page need to be rebased and re-integrated with the new 53-feature inference pipeline.

Justification: Confidence scoring (Issue #46) was a client-requested feature completed in Sprint 4. The predictions page was the primary user interface for generating forecasts. Both were lost during a large merge that refactored the inference module.

Impact: Dante's `dante_feature` branch (PR #45) must be rebased onto the new `main`. The `confidence.py` module needs to be adapted for the new feature names and pipeline. The predictions page needs to work with the updated `PredictionResponse` type.

Approval: Approved by team consensus on March 4, 2026. Implemented during Sprint 5 (US-30, US-31).

Change Request #4

Field	Details
CR ID	CR-004
Title	Migrate from JSON model format to native XGBoost .ubj

Requested By	Harsh Kumar (Product Owner)
Date Submitted	March 6, 2026
Priority	Medium
Status	Approved & Implemented

Description: Switch model artifact storage from XGBoost JSON export to native `.ubj` binary format with an accompanying `meta.json` file containing model metadata (horizon, split date, feature names, etc.).

Justification: Native format loads significantly faster (~1.2s vs ~3s for JSON), uses less disk space, and the `meta.json` sidecar provides structured metadata for the model loader.

Impact: Updated `model_loader.py` to read native format. New artifact directory structure with `meta.json`. Training pipeline updated to export in new format.

Approval: Approved by Product Owner on March 6, 2026. Implemented by Harsh Kumar.

Change Request #5

Field	Details
CR ID	CR-005
Title	Add Streamlit as secondary frontend
Requested By	Harsh Kumar (Product Owner)
Date Submitted	March 8, 2026
Priority	Medium
Status	Approved & Implemented

Description: Add a Streamlit-based frontend application alongside the existing React dashboard, focused on data exploration, inference testing, and dataset snapshot management.

Justification: Data analysts and ML engineers need a quick way to explore market data and test predictions without the full React UI. Streamlit provides rapid prototyping for data-focused workflows.

Impact: New `frontend_streamlit/` directory with `app.py`. Runs independently from the React frontend. No changes to existing React code.

Approval: Approved by team on March 8, 2026. Merged via PR #49.

13. User Support Documentation

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Table of Contents

1. User's Guide
2. Online Help
3. Release Notes
4. Training Materials

1. User's Guide

A comprehensive User's Manual has been prepared as a separate document (see `Documentation/Sprint4/Users_Manual.pdf`). The manual covers all aspects of using MarketSight from an end-user perspective:

- Getting Started and Navigation
- Dashboard Overview with summary cards
- Browsing and Searching Stocks
- Stock Detail Charts (candlestick/line, time ranges, prediction overlay)
- Generating Predictions with confidence scores
- Understanding Confidence Scores (color coding and interpretation)
- Account Settings and Password Management
- Admin Panel for user management
- Appearance themes (light/dark/system)
- Troubleshooting FAQ and Glossary

2. Online Help

The application includes built-in help through:

- FastAPI auto-generated API documentation at `/docs` (Swagger UI) and `/redoc` (ReDoc)
- Streamlit app with built-in page descriptions and tooltips
- Tooltip hints on chart controls and prediction cards

- Error messages with actionable descriptions (e.g., "Stock not found", "Insufficient data")
- Server setup documentation at `docs_server_setup.md`

3. Release Notes - v1.1.0 (Sprint 5 IOC Release)

New Features:

- Streamlit frontend for market data exploration, inference testing, and dataset management
- DRI server production deployment with management scripts and SSL
- Native XGBoost model format (.ubj) with meta.json metadata
- Centralized feature engineering module for training/inference parity
- Data warehouse sync scripts (remote PostgreSQL to local via SSH tunnel)
- Stock detail page UI improvements with chart range breaks
- UTC datetime standardization across all frontend displays

Restored Features (from Sprint 4 merge conflict):

- Prediction confidence scoring re-integrated with 53-feature pipeline
- Full predictions page with stock selector, Predict/Predict All, confidence display

Known Limitations:

- No user authentication (open access)
- No real-time WebSocket streaming
- No prediction history or accuracy tracking

4. Training Materials

- **User's Manual:** See `Documentation/Sprint4/Users_Manual.pdf` for end-user guide
- **Development Setup Guide:** See `development.md` for Docker Compose setup
- **DRI Server Setup:** See `docs_server_setup.md` for production server instructions
- **DRI Documentation:** See `Documentation/DRI.md` for HPC server access
- **API Reference:** Access `/docs` endpoint for interactive API documentation

14. Developer's Guide (Final)

Prepared by: Zane Tessmer | **Last Updated:** March 17, 2026

Final Architecture Diagram

```
+------------------+     +-------------------+     +------------------+
|    Frontend      |     |     Backend       |     |    Database      |
|  React 19 + TS   |<--->|  FastAPI + Python  |<--->|  PostgreSQL 16   |
|  Vite + Tailwind |     |  SQLAlchemy 2.0   |     |  market schema   |
|  TanStack Router |     |  XGBoost (native) |     |  stocks + prices |
+------------------+     +-------------------+     +------------------+
        |                        |
+------------------+     +------+------+
| Streamlit App    |     |   Airflow   |
| Data Explorer    |     | Scheduler   |
| Inference Test   |     | DAGs: seed, |
| Dataset Mgmt     |     | retrain     |
+------------------+     +-------------+
        |                        |
  +-----+------+         +------+------+
  |   Nginx    |         | DRAC HPC    |
  |   Proxy    |         | GPU Train   |
  |   + SSL    |         | Optuna HPO  |
  +------------+         +------------+
        |
  +-----+------+
  | DRI Server |
  | Production |
  +------------+
```

Final Use Case Diagram

```
                    +---------------------------+
                    |       MarketSight         |
                    +---------------------------+
                    |                           |
  +------+         | [View Stock List]         |
  | User |-------->| [View Stock Chart]        |
  +------+         | [Get Prediction]          |
```

```
      |            | [Predict All Stocks]      |
      |            | [View Dashboard]          |
      |            | [Manage Settings]         |
      |            +---------------------------+
      |
  +-------+        +---------------------------+
  | Admin |------->| [Manage Users]            |
  +-------+        | [Add/Edit/Delete Users]   |
                   +---------------------------+
                           |
  +-----------+    +---------------------------+
  | Data      |--->| [Explore Market Data]     |
  | Analyst   |    | [Run Inference Test]      |
  | (Streamlit)|   | [Manage Dataset Snapshots]|
  +-----------+    +---------------------------+
                           |
  +-----------+    +---------------------------+
  | Airflow   |--->| [Seed Market Data]        |
  | Scheduler |    | [Retrain ML Model]        |
  +-----------+    | [Snapshot Datasets]       |
                   +---------------------------+
```

Project Code Description

Module	Path	Description
API Layer	`backend/app/api/`	FastAPI routers, dependency injection, health check
Market Module	`backend/app/modules/market/`	Stock data models, CRUD (raw SQL optimized), service, API
Inference Module	`backend/app/modules/inference/`	ML prediction: model loading (.ubj), 53-feature eng, confidence, API
Training Module	`backend/app/modules/training/`	Training job API: start, monitor, stream logs
Data Module	`backend/app/modules/data/`	Data snapshot API: build, list, download datasets
Core Config	`backend/app/core/`	Pydantic settings, database engine, security utilities
Frontend Routes	`frontend/src/routes/`	TanStack Router: dashboard, stocks, predictions, training monitor
Frontend Components	`frontend/src/components/`	Charts, tables, admin forms, settings, sidebar
Frontend Services	`frontend/src/services/`	API client functions for market and inference endpoints
Streamlit App	`frontend_streamlit/app.py`	Streamlit: market data, inference, dataset management
ML Scripts	`ml/scripts/`	Data prep, training, inference scripts organized by function
ML Features	`ml/features/`	53-feature engineering pipeline (RSI, MACD, Bollinger, lag, volume)
ML Models	`ml/models/`	Model artifacts: native .ubj format + meta.json

Airflow DAGs	`airflow/dags/`	Data seeding, model retraining, connectivity test
DW Sync	`scripts/sync_dw_to_local_postgres.py`	Remote-to-local PostgreSQL sync with SSH tunnel
Server Docs	`docs_server_setup.md`	DRI production server setup and management

Testing Code Description

Test File	Type	Description
`tests/api/test_utils.py`	Unit	Health check endpoint verification
`tests/modules/test_market.py`	Integration	Full market API request/response cycle
`tests/modules/test_market_crud.py`	Unit	Database CRUD operations for stocks and prices
`tests/modules/test_market_service.py`	Unit	MarketService business logic layer
`tests/modules/test_inference.py`	Integration	Inference API with mocked model pipeline
`tests/modules/test_confidence.py`	Unit	Confidence scoring sub-functions and edge cases
`tests/conftest.py`	Fixture	Shared DB session and FastAPI TestClient setup
`tests/modules/conftest.py`	Fixture	Market test data: stocks + OHLC seeding

Continuous Integration Report

Pipeline: GitHub Actions (`ci.yml`) **Trigger:** Push or PR to `main` branch **Last Run:** Passing (all 6 jobs green)

Job	Status	Duration
backend-lint	Passing	~30s
backend-typecheck	Passing	~45s
backend-test	Passing	~2min
frontend-lint	Passing	~20s
frontend-build	Passing	~45s
docker-build	Passing	~3min

Total pipeline time: ~7 minutes **Test results:** All tests passing (market CRUD, market API, market service, inference API, confidence scoring, health check)

Part C - Testing & Acceptance

15. Acceptance Test Results

Tested by: Zane Tessmer, Dante Bertolutti **Date:** March 15, 2026

Acceptance Test Cases

Test ID	Test Description	Expected Result	Actual Result	Status
AT-01	Navigate to /dashboard and verify summary cards load	4 summary cards displayed (stocks, sectors, model, API)	Cards displayed correctly	PASS
AT-02	Navigate to /dashboard/stocks and search for "AAPL"	AAPL stock appears in filtered list	AAPL found and displayed	PASS
AT-03	Click AAPL stock and verify candlestick chart renders	Interactive candlestick chart with OHLC data	Chart rendered with range breaks	PASS
AT-04	Toggle chart to line mode	Chart switches to line series	Line chart displayed	PASS
AT-05	Select 1W time range on chart	Chart zooms to last 7 days of data	Time range applied correctly	PASS
AT-06	Enable prediction overlay on chart	Dashed orange line from last price to predicted price	Overlay rendered correctly	PASS
AT-07	Navigate to /dashboard/predictions	Predictions page with stock selector loads	Page loaded correctly	PASS
AT-08	Select AAPL and get prediction	Prediction card shows symbol, prices, return %, confidence	All fields populated correctly	PASS
AT-09	Verify confidence score is between 0 and 1	Score displayed as percentage (0-100%)	Score within valid range	PASS
AT-10	Click "Predict All" button	Predictions generated for all active stocks	Prediction cards displayed for all stocks	PASS
AT-11	Verify confidence color coding	Green > 70%, Yellow 40-70%, Red < 40%	Colors matched thresholds	PASS
AT-12	Hit /api/v1/inference/predict/INVALID	404 error returned	404 "Stock not found"	PASS
AT-13	Hit /api/v1/utils/health-check/	Returns true with 200 status	Response: true, status: 200	PASS
AT-14	Run full CI pipeline on main branch	All 6 jobs pass	All jobs green	PASS

AT-15	Docker compose up and verify all services start	All services running and healthy	All services running	PASS
AT-16	Verify Training Monitor page loads at /dashboard/training	Training monitor page with log viewer	Page loaded correctly	PASS
AT-17	Verify Streamlit app loads and shows market data	Streamlit app with stock data	App loaded, data displayed	PASS
AT-18	Verify datetime displays are in UTC across all pages	All timestamps show UTC format	UTC formatting consistent	PASS
AT-19	Verify model loads in native .ubj format	Model loaded within 2s, meta.json parsed	Model loaded in ~1.2s	PASS
AT-20	Verify DRI server deployment serves the application	Application accessible via production URL	App served correctly with SSL	PASS

Acceptance Sign-Off

All 20 acceptance test cases passed. The system meets the Sprint 5 acceptance criteria for the IOC (Initial Operational Capability) release.

Signed:

Client Representative: _____ Date: _____

Product Owner (Harsh Kumar): _____ Date: _____

Scrum Master (Zane Tessmer): _____ Date: _____